

# Objects, Files, and ArrayLists

Reading three file formats, and writing one back out

---

**A word before you read.** These are my notes from over the years, cleaned up so they are readable. A few things to keep in mind:

- **They are not a study guide, and they are not gospel.** Not everything here appears in a lecture, and not every topic has notes yet; I will write more as we go.
- **I write from memory and revise every semester.** A typo or an imprecise statement can survive longer than it should.
- **I use AI to help** convert documents to markdown, check for issues, and add explanations. I do not trust it completely, so I am still editing these myself.
- **Java changes.** The basics we cover are pretty resilient, but some features do move over time.

So: if you find something that looks wrong, contradicts the book, or just does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. When in doubt, the textbook is the tiebreaker, and the compiler beats us both.

A quick note before starting: today's demo uses a small custom class, **Racer**, to hold a few pieces of data together. This is meant as a light, practical introduction, just enough to build and use objects in this week's lab. Classes and objects get covered in full depth in a later class, so do not worry about understanding everything about how or why a class works yet, only how to create one and call its methods.

With that said, today's goal is to see the same **Racer** objects handled several different ways: built by hand, read from a plain space-separated text file, read from a CSV file two different ways, and finally written back out to a file. The **Racer** class and its **ArrayList<Racer>** stay the same throughout; only where the data comes from, or goes to, changes.

## 1. Project Setup

Do this once, before Part 1, and everything below will just work.

1. Create a new Eclipse project.
2. Right click the **src** folder, choose **New > Package**, and name it **racer**. Every class today goes in this package, starting with **package racer**; as its first line.
3. Right click the project name, choose **New > Folder**, and name it **files**. Every data file you read today, and everything your code writes out, lives here.

## 2. Part 1: Building Racer Objects by Hand

A class is a blueprint. It describes what a `Racer` object knows about itself (its name, its vehicle, and its top speed) but it does not create any actual racers on its own. Each time the program writes `new Racer(...)`, one new object gets built from that blueprint and stored in memory.

### The Racer class

```
package racer;

public class Racer {
    // Fields: what every Racer object knows about itself.
    private String name;
    private String vehicle;
    private double topSpeed;

    // Constructor: fills in the fields from the values passed in.
    public Racer(String name, String vehicle, double topSpeed) {
        this.name = name;
        this.vehicle = vehicle;
        this.topSpeed = topSpeed;
    }

    // Getters: hand back one field at a time.
    public String getName() {
        return name;
    }

    public String getVehicle() {
        return vehicle;
    }

    public double getTopSpeed() {
        return topSpeed;
    }
}
```

The constructor is the piece that runs at the exact moment `new Racer(...)` executes. Its only job is to take the values passed in and store them in the object's fields, using `this.name = name`; to tell Java which `name` is the field and which is the parameter.

### Creating racers and putting them in a list

```
package racer;

import java.util.ArrayList;

public class RacerDemoPart1 {
    public static void main(String[] args) {
        ArrayList<Racer> racers = new ArrayList<Racer>();

        // Build an object, then add it to the list, in one step.
        racers.add(new Racer("Mario", "Standard Kart", 62.5));
        racers.add(new Racer("Bowser", "Heavy Cruiser", 68.0));
        racers.add(new Racer("Toad", "Light Buggy", 58.0));
    }
}
```

```

    // Loop through the list, calling a getter on each object.
    for (int i = 0; i < racers.size(); i++) {
        Racer r = racers.get(i);
        System.out.println(r.getName() + " drives the " + r.getVehicle()
            + " at " + r.getTopSpeed() + " mph");
    }

    // Walk the list, keeping the best one seen so far.
    Racer fastest = racers.get(0);
    for (int i = 1; i < racers.size(); i++) {
        if (racers.get(i).getTopSpeed() > fastest.getTopSpeed()) {
            fastest = racers.get(i);
        }
    }
    System.out.println("Fastest: " + fastest.getName());
}
}

```

Every call to `racers.add(new Racer(...))` builds one object and adds it to the list in a single step. Once the objects are in the `ArrayList`, `racers.get(i)` hands back one `Racer` object at a time, and calling `.getName()` or `.getTopSpeed()` on it reads that object's fields. The fastest-racer loop at the bottom does not use anything special to search the list, it just walks every element with a regular for loop and keeps track of the best one seen so far, an idea that comes back later today.

### 3. Part 2: Reading a Space-Separated File

Not every data file is a CSV. `lapTimes.txt` below has one racer per line, with a name and a lap time separated by a space instead of a comma.

#### lapTimes.txt

```

Mario 78.2
Bowser 81.5
Toad 76.9

```

#### Reading lapTimes.txt

```

package racer;

import java.io.File;
import java.util.Scanner;

public class RacerDemoPart2 {
    public static void main(String[] args) throws Exception {
        Scanner fileScanner = new Scanner(new File("files/lapTimes.txt"));

        String fastestName = "";
        double fastestTime = Double.MAX_VALUE;

        // next() and nextDouble() automatically stop at whitespace, so
        // no manual splitting is needed for a space-separated file.
        while (fileScanner.hasNext()) {
            String name = fileScanner.next();
            double lapTime = fileScanner.nextDouble();

```

```

        System.out.println("Best lap for " + name + ": " + lapTime + " seconds");

        if (lapTime < fastestTime) {
            fastestTime = lapTime;
            fastestName = name;
        }
    }
    fileScanner.close();

    System.out.println("Fastest lap: " + fastestName);
}
}

```

`Scanner` already knows how to split on whitespace, so no manual splitting is needed here. Calling `.next()` reads the next word, and `.nextDouble()` reads the next number, each one stopping automatically at the next space or line break. This is the same idea you will use to read a file with several numbers per line, separated by spaces, later this week.

## 4. Part 3: Reading a CSV File Without a Library

`racers.csv` below is the same three racers as Part 1, this time saved to a file with commas between the fields.

### `racers.csv`

```

Name,Vehicle,TopSpeed
Mario,Standard Kart,62.5
Bowser,Heavy Cruiser,68.0
Toad,Light Buggy,58.0

```

### Reading `racers.csv` by hand

```

package racer;

import java.io.File;
import java.util.Scanner;
import java.util.ArrayList;

public class RacerDemoPart3 {
    public static void main(String[] args) throws Exception {
        ArrayList<Racer> racers = new ArrayList<Racer>();
        Scanner fileScanner = new Scanner(new File("files/racers.csv"));

        fileScanner.nextLine(); // skip the header row

        while (fileScanner.hasNextLine()) {
            String line = fileScanner.nextLine();
            String[] parts = line.split(","); // splits at every comma

            String name = parts[0];
            String vehicle = parts[1];
            double topSpeed = Double.parseDouble(parts[2]);

```

```

        racers.add(new Racer(name, vehicle, topSpeed));
    }
    fileScanner.close();

    for (int i = 0; i < racers.size(); i++) {
        System.out.println(racers.get(i).getName());
    }
}

```

This reads each line as one long `String` and then calls `line.split(",")` to cut it into pieces wherever a comma appears. It works here because nothing in `racers.csv` contains a comma inside a field. If a vehicle name were written as "Kart, Deluxe", with a comma of its own, `split(",")` would cut that name into two pieces and shift every value after it over by one column, silently producing the wrong data instead of an error. Real-world CSV data is often messier than today's example, which is why a dedicated library is worth using.

## 5. Part 4: Reading the Same CSV File with OpenCSV

This part reads the exact same `racers.csv` from Part 3 again, using the OpenCSV library instead of `split(",")`, so you can compare the two approaches directly.

### Adding OpenCSV and Apache Commons Lang to your Eclipse project

OpenCSV needs one small helper library, Apache Commons Lang, to run. The steps below are a simple way to add both jars to a project by hand. This is not necessarily the most robust way to manage a library in a larger project, but it is enough for today, and it will be walked through live in class.

1. Download OpenCSV from <https://sourceforge.net/projects/opencsv/>, and Apache Commons Lang from [https://commons.apache.org/proper/commons-lang/download\\_lang.cgi](https://commons.apache.org/proper/commons-lang/download_lang.cgi). Each download is an archive; extract both and find the plain `.jar` file in each one, not the `-sources` or `-javadoc` jar.
2. In Eclipse, right click your project in the Package Explorer, choose **New > Folder**, and name it `lib`. Copy both jar files into that folder.
3. Right click the project name, choose **Build Path > Configure Build Path**, open the **Libraries** tab, choose **Classpath**, then **Add JARs** and select both jars in your `lib` folder.
4. Click **Apply and Close**. The line `import com.opencsv.CSVReader;` should no longer show an error.

### Reading `racers.csv` with OpenCSV

```

package racer;

import com.opencsv.CSVReader;
import java.io.FileReader;
import java.util.ArrayList;

public class RacerDemoPart4 {
    public static void main(String[] args) throws Exception {

```

```

    ArrayList<Racer> racers = new ArrayList<Racer>();

    // CSVReader hands back one line at a time as a String array,
    // already split on the commas (even commas inside quotes).
    CSVReader reader = new CSVReader(new FileReader("files/racers.csv"));
    reader.readNext(); // skip the header row

    String[] line;
    while ((line = reader.readNext()) != null) {
        String name = line[0];
        String vehicle = line[1];
        double topSpeed = Double.parseDouble(line[2]);
        racers.add(new Racer(name, vehicle, topSpeed));
    }
    reader.close();

    // racers is an ArrayList<Racer>, same as Part 1; print it the same way.
    for (int i = 0; i < racers.size(); i++) {
        System.out.println(racers.get(i).getName());
    }
}

```

`reader.readNext()` returns one line at a time as a `String` array, already correctly split at the commas, handling a comma inside a quoted field the way `split(",")` could not. It returns `null` once the file runs out, which is what the `while` loop's condition is checking.

## 6. Part 5: Writing Data to a File

Reading data is only half the picture. This part builds the same racer list from Part 1 again and writes a short report out to a new file, `raceReport.txt`.

### Writing `raceReport.txt`

```

package racer;

import java.io.FileWriter;
import java.io.PrintWriter;
import java.util.ArrayList;

public class RacerDemoPart5 {
    public static void main(String[] args) throws Exception {
        // Rebuilding the same three racers from Part 1, just to have
        // something to write out below.
        ArrayList<Racer> racers = new ArrayList<Racer>();
        racers.add(new Racer("Mario", "Standard Kart", 62.5));
        racers.add(new Racer("Bowser", "Heavy Cruiser", 68.0));
        racers.add(new Racer("Toad", "Light Buggy", 58.0));

        // PrintWriter has the same println(...) method as System.out,
        // just aimed at a file instead of the console.
        PrintWriter writer = new PrintWriter(new FileWriter("files/raceReport.txt"));

        for (int i = 0; i < racers.size(); i++) {
            Racer r = racers.get(i);

```

```
        writer.println(r.getName() + " - " + r.getVehicle() + " - " + r.getTopSpeed() + "
    mph");
    }

    writer.close(); // required, or the file can end up empty
}
}
```

`PrintWriter` has the same `println(...)` method you have already used with `System.out`, except its output goes to a file instead of the console. The call to `writer.close()` at the end is not optional: output can stay held in memory rather than actually written to disk until the writer is closed, so skipping it can leave the file empty or cut off partway through.

## 7. What to Notice

- The `Racer` class did not change across any of the five parts above. Objects do not care whether the values used to build them were typed directly into the code, read from a file, or about to be written back out to one.
- `Scanner`'s `.next()`, `.nextInt()`, and `.nextDouble()` methods split on whitespace automatically, which is exactly what a space-separated file needs.
- `split(",")` is fine for simple, well-behaved CSV data, but it breaks the moment a field contains a comma of its own. `OpenCSV` exists to handle that case correctly.
- Writing to a file with `PrintWriter` looks almost identical to printing to the console, the output just goes somewhere else, and the file has to be closed when you are done.

**Looking ahead.** The lab that pairs with this demo asks you to combine two files into one list, matching values by an ID and updating objects that already exist in the list rather than only adding new ones. Everything above shows how to read and write each kind of file on its own; combining and updating come next.